

Nama : Fadia Farah Sabila

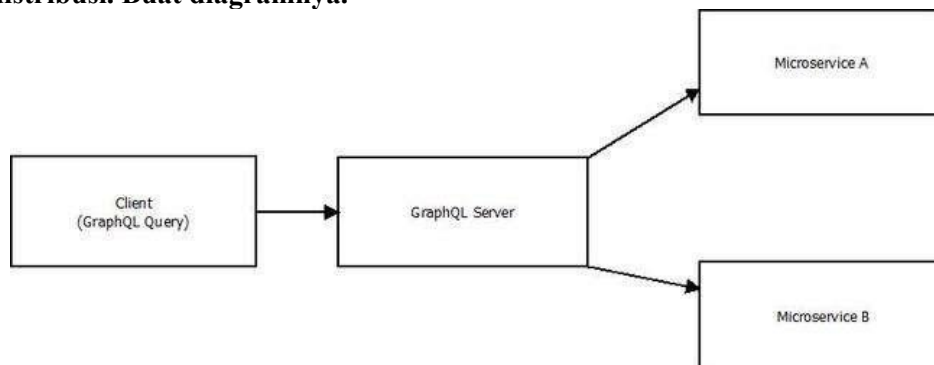
NIM : 245410046

1. **Jelaskan teorema CAP dan BASE dan keterkaitan keduanya. Jelaskan menggunakan contoh yang pernah anda gunakan.**

Teorema CAP Teorema CAP menyatakan bahwa dalam sistem terdistribusi, kita tidak bisa memiliki ketiga aspek berikut secara bersamaan: Consistency (Konsistensi): Semua node memiliki data yang sama. Availability (Ketersediaan): Sistem selalu dapat diakses dan memberikan respons. Partition Tolerance (Toleransi Partisi): Sistem tetap berfungsi meskipun ada partisi jaringan. Contoh : Sistem e-commerce dengan banyak node yang melayani ribuan transaksi per detik. Jika terjadi pemisahan jaringan antara beberapa server, sistem harus memilih antara konsistensi data atau tetap memberikan layanan (availability). BASE adalah pendekatan yang lebih fleksibel untuk mengatasi keterbatasan CAP: Basically Available (Ketersediaan Dasar): Sistem tetap bisa diakses walau tidak semua node sinkron. Soft-state (Keadaan Lembut): Node dapat memiliki data yang berbeda sementara. Eventually Consistent (Konsistensi Akhir): Sistem akan mencapai konsistensi pada akhirnya. Contoh: Sistem e-commerce tetap menerima transaksi meskipun data pada beberapa node belum konsisten. Nanti sistem melakukan sinkronisasi sehingga data akhirnya konsisten.

Keterkaitan CAP dan BASE

- CAP menekankan **trade-off** antara konsistensi, ketersediaan, dan toleransi partisi.
 - BASE adalah pendekatan pragmatis untuk **mengutamakan ketersediaan dan toleransi partisi**, dengan konsistensi yang dicapai **secara bertahap**.
2. **Jelaskan keterkaitan antara GraphQL dengan komunikasi antar proses pada sistem terdistribusi. Buat diagramnya.**



Keterangan:

- Client mengirim query ke GraphQL Server.
- GraphQL Server memproses query dan meneruskannya ke microservice terkait.
- Microservice merespons data kembali ke GraphQL Server, lalu diteruskan ke Client.
- Dengan ini, komunikasi antar proses tetap efisien dan terstruktur.

3. **Dengan menggunakan Docker / Docker Compose, buatlah streaming replication di PostgreSQL yang bisa menjelaskan sinkronisasi. Tulislah langkah-langkah pengerjaannya dan buat penjelasan secukupnya.**

Streaming Replication di PostgreSQL Windows

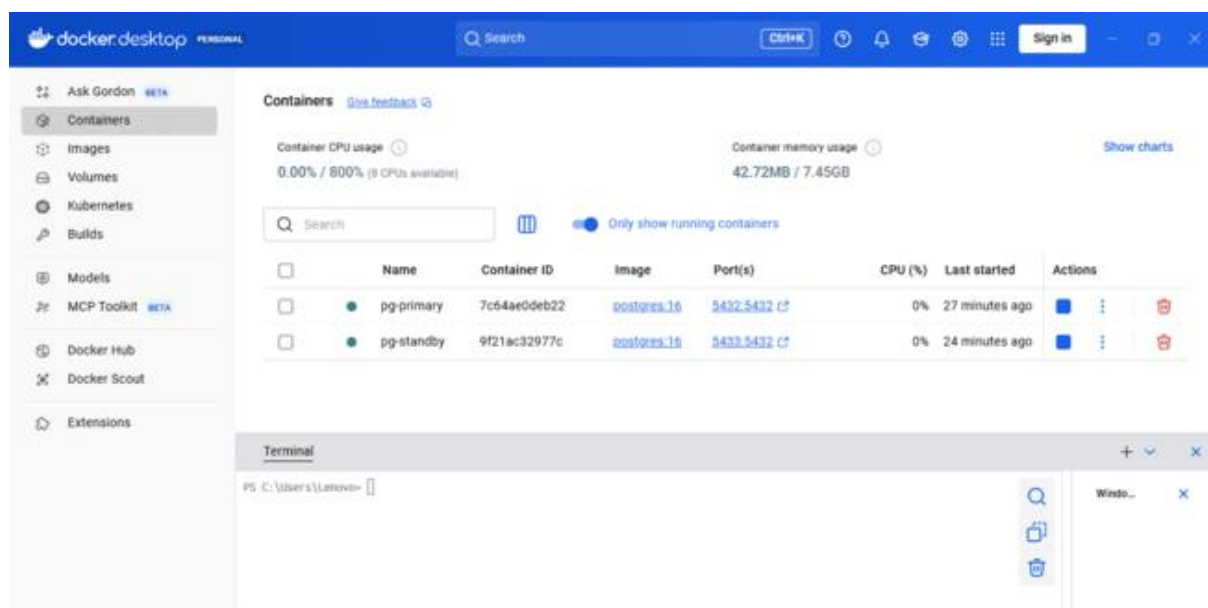
<https://www.crunchydata.com/blog/postgres-streaming-replication-on-windows-a-quick-guide>

Persiapan Awal

Aktifkan Docker Desktop:

Struktur Direktori dan File

```
PS C:\Windows\system32> docker exec -it pg-primary bash
root@7c64ae0deb22:/# ls -l var/lib/postgresql/data/
total 128
drwx----- 6 postgres postgres 4096 Oct 27 17:45 base
drwx----- 2 postgres postgres 4096 Oct 27 17:44 global
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_commit_ts
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_dynshmem
-rw----- 1 postgres postgres 5794 Oct 27 17:43 pg_hba.conf
-rw----- 1 postgres postgres 2640 Oct 27 17:40 pg_ident.conf
drwx----- 4 postgres postgres 4096 Oct 27 17:55 pg_logical
drwx----- 4 postgres postgres 4096 Oct 27 17:40 pg_multixact
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_notify
drwx----- 3 postgres postgres 4096 Oct 27 17:44 pg_replslot
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_serial
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_snapshots
drwx----- 2 postgres postgres 4096 Oct 27 17:43 pg_stat
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_stat_tmp
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_subtrans
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_tblspc
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_twophase
-rw----- 1 postgres postgres    3 Oct 27 17:40 PG_VERSION
drwx----- 3 postgres postgres 4096 Oct 27 17:50 pg_wal
drwx----- 2 postgres postgres 4096 Oct 27 17:40 pg_xact
-rw----- 1 postgres postgres   88 Oct 27 17:40 postgresql.auto.conf
-rw----- 1 postgres postgres 29890 Oct 27 17:43 postgresql.conf
-rw----- 1 postgres postgres   36 Oct 27 17:43 postmaster.opts
-rw----- 1 postgres postgres   94 Oct 27 17:43 postmaster.pid
root@7c64ae0deb22:/#
```



docker

Windows PowerShell

```
PS C:\Users\Lenovo> # Versi Postgres
PS C:\Users\Lenovo> $PG_VERSION = "postgres:16"
PS C:\Users\Lenovo>
PS C:\Users\Lenovo> # Kredensial
PS C:\Users\Lenovo> $POSTGRES_PASSWORD = "supersecretpassword123"
PS C:\Users\Lenovo> $REPL_USER = "repl_user"
PS C:\Users\Lenovo> $REPL_PASSWORD = "supersecretpassword456"
PS C:\Users\Lenovo>
PS C:\Users\Lenovo> # Nama-nama Objek Docker
PS C:\Users\Lenovo> $NETWORK_NAME = "pg-replication-net"
PS C:\Users\Lenovo> $PRIMARY_HOST = "pg-primary"
PS C:\Users\Lenovo> $STANDBY_HOST = "pg-standby"
PS C:\Users\Lenovo> $PRIMARY_DATA_VOL = "pg-primary-data"
PS C:\Users\Lenovo> $STANDBY_DATA_VOL = "pg-standby-data"
```

Tentukan Variabel

```
PS C:\Users\Lenovo> docker network create $NETWORK_NAME
6609bb1df5171fdbb54df343751271e5fcd9df1848ea22f272bc78b239296a64
PS C:\Users\Lenovo> docker volume create $PRIMARY_DATA_VOL
pg-primary-data
PS C:\Users\Lenovo> docker volume create $STANDBY_DATA_VOL
pg-standby-data
```

membuat Jaringan dan Volume Docker

Jalankan Node Primary

```
PS C:\Users\Lenovo> docker run -d `
>> --name $PRIMARY_HOST `
>> --network $NETWORK_NAME `
>> -e POSTGRES_PASSWORD=$POSTGRES_PASSWORD `
>> -v "${PRIMARY_DATA_VOL}:/var/lib/postgresql/data" `
>> -p 5432:5432 `
>> $PG_VERSION
Unable to find image 'postgres:16' locally
16: Pulling from library/postgres
6a570b5ed774: Pull complete
85ec668df6b0: Pull complete
147d1d7c0318: Pull complete
e69f32e226f6: Pull complete
f50de807d554: Pull complete
8221e00ace81: Pull complete
bc405403221b: Pull complete
e7fca4723572: Pull complete
45e682a0ad62: Pull complete
91264d87777d: Pull complete
bd122ab2336d: Pull complete
b9fcf05c0044: Pull complete
82742b6e768c: Pull complete
Digest: sha256:4eb532412200f7bbf15d62ee0d96e020a9eae9eae76066692474a7371c4d83
Status: Downloaded newer image for postgres:16
7c64ae0deb2250473d9b1973a85f651410c4ab4ec67a3d2a8acc894cea964f4b
```

Konfigurasi Node Primary untuk Replikasi

```
PS C:\Users\Lenovo> Start-Sleep -seconds 20
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "CREATE ROLE $REPL_USER LOGIN REPLICATION ENCRYPTED PASSWORD '$REPL_PASSWORD' ;"
```

Ubah Konfigurasi **postgresql.conf**:


```

PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "wal_level = replica" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "max_wal_senders = 4" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "max_replication_slots = 4" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "hot_standby = on" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "wal_log_hints = on" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "hot_standby_feedback = on" >> /var/lib/postgresql/data/postgresql.conf'
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "wal_replication = $REPL_USER 0.0.0.0/0 stream-sha-256" >> /var/lib/postgresql/data/pg_hba.conf'
PS C:\Users\Lenovo> docker restart $PRIMARY_HOST
pg-primary

```

Ubah Konfigurasi `pg_hba.conf`: Restart Node Primary

```

PS C:\Users\Lenovo> docker exec $PRIMARY_HOST bash -c 'echo "host replication $REPL_USER 0.0.0.0/0 stream-sha-256" >> /var/lib/postgresql/data/pg_hba.conf'
PS C:\Users\Lenovo> docker restart $PRIMARY_HOST
pg-primary

```

Buat Slot Replikasi:

```

PS C:\Users\Lenovo> Start-Sleep -Seconds 10
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "SELECT * FROM pg_create_physical_replication_slot('standby1');"
 slot_name | lsn
-----
 standby1 |
(1 row)

```

Cloning Database ke Node Standby

```

PS C:\Users\Lenovo> docker run --rm `
>> -e PGPASSWORD=$REPL_PASSWORD `
>> --network $NETWORK_NAME `
>> -v "${STANDBY_DATA_VOL}:/var/lib/postgresql/data" `
>> $PG_VERSION `
>> pg_basebackup -h $PRIMARY_HOST -U $REPL_USER -D /var/lib/postgresql/data -Fp -Xs -R -c fast -v -P `
>> -S 'standby1'
pg_basebackup: initiating base backup, waiting for checkpoint to complete
pg_basebackup: checkpoint completed
pg_basebackup: write-ahead log start point: 0/2000028 on timeline 1
pg_basebackup: starting background WAL receiver
23246/23246 kB (100%), 0/1 tablespace (...ostgresql/data/global/pg_control)
23246/23246 kB (100%), 1/1 tablespace
pg_basebackup: write-ahead log end point: 0/2000100
pg_basebackup: waiting for background process to finish streaming ...
pg_basebackup: syncing data to disk ...
pg_basebackup: renaming backup_manifest.tmp to backup_manifest
pg_basebackup: base backup completed

```

Jalankan Node Standby (Database Cadangan)

```

PS C:\Users\Lenovo> docker run -d `
>> --name $STANDBY_HOST `
>> --network $NETWORK_NAME `
>> -e POSTGRES_PASSWORD=$POSTGRES_PASSWORD `
>> -v "${STANDBY_DATA_VOL}:/var/lib/postgresql/data" `
>> -p 5433:5432 `
>> $PG_VERSION
9f21ac32977c487e047fe6c6eaa210f874d40ca24fe6aa8f78cf8feee51881af

```

Uji Coba dan Verifikasi Replikasi

```

PS C:\Users\Lenovo> docker logs $STANDBY_HOST
PostgreSQL Database directory appears to contain a database; Skipping initialization

2025-10-27 17:46:12.500 UTC [1] LOG:  starting PostgreSQL 16.10 (Debian 16.10-1.pgdg13-1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2025-10-27 17:46:12.501 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2025-10-27 17:46:12.501 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2025-10-27 17:46:12.514 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-10-27 17:46:12.534 UTC [29] LOG:  database system was interrupted; last known up at 2025-10-27 17:45:53 UTC
2025-10-27 17:46:13.120 UTC [29] LOG:  entering standby mode
2025-10-27 17:46:13.121 UTC [29] LOG:  starting backup recovery with redo LSN 0/2000028, checkpoint LSN 0/2000060, on timeline ID 1
2025-10-27 17:46:13.138 UTC [29] LOG:  redo starts at 0/2000028
2025-10-27 17:46:13.147 UTC [29] LOG:  completed backup recovery with redo LSN 0/2000028 and end LSN 0/2000100
2025-10-27 17:46:13.147 UTC [29] LOG:  consistent recovery state reached at 0/2000100
2025-10-27 17:46:13.147 UTC [1] LOG:  database system is ready to accept read-only connections
2025-10-27 17:46:13.197 UTC [30] LOG:  started streaming WAL from primary at 0/3000000 on timeline 1

```

Cek Status di Primary:

```

PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "SELECT * FROM pg_stat_replication;"
 pid | usename | application_name | client_addr | client_hostname | client_port |
-----+-----+-----+-----+-----+-----+
 47 | repl_user | walreceiver | 172.20.0.3 | | 50176 |
(1 row)

PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "CREATE TABLE IF NOT EXISTS test_rep (id INT PRIMARY KEY, message TEXT);"
CREATE TABLE
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "INSERT INTO test_rep (id, message) VALUES (1, 'Hello dari Primary!') ON CONFLICT (id) DO NOTHING;"
INSERT 0 1
PS C:\Users\Lenovo> Start-Sleep -Seconds 3
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "SELECT * FROM test_rep;"
 id | message
--|-----
 1 | Hello dari Primary!
(1 row)

```

status **streaming**

Buat Data di Primary:

```
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "CREATE TABLE IF NOT EXISTS test_rep (id INT PRIMARY KEY, message TEXT);"
CREATE TABLE
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "INSERT INTO test_rep (id, message) VALUES (1, 'Halo dari Primary!') ON CONFLICT (id) DO NOTHING;"
INSERT 0 1
PS C:\Users\Lenovo> Start-Sleep -Seconds 3
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "SELECT * FROM test_rep;"
 id | message
-----+-----
  1 | Halo dari Primary!
(1 row)
```

Baca Data di Standby:

```
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "SELECT * FROM test_rep;"
 id | message
-----+-----
  1 | Halo dari Primary!
(1 row)
```

Tes Gagal Tulis di Standby

```
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "INSERT INTO test_rep (id, message) VALUES (2, 'Halo dari Standby!');"
ERROR: cannot execute INSERT in a read-only transaction
PS C:\Users\Lenovo> Write-Host "--- (PRIMARY) Membuat tabel 'mahasiswa' ---"
--- (PRIMARY) Membuat tabel 'mahasiswa' ---
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "CREATE TABLE mahasiswa (nim VARCHAR(10) PRIMARY KEY, nama VARCHAR(100), jurusan VARCHAR(50));"
CREATE TABLE
PS C:\Users\Lenovo> Write-Host "--- (PRIMARY) Menginput 3 data mahasiswa ---"
--- (PRIMARY) Menginput 3 data mahasiswa ---
```

skenario kasus yang sempurna untuk menguji replikasi.

Buat Tabel **mahasiswa** di PRIMARY

```
PS C:\Users\Lenovo> Write-Host "--- (PRIMARY) Menginput 3 data mahasiswa ---"
--- (PRIMARY) Menginput 3 data mahasiswa ---
PS C:\Users\Lenovo> docker exec $PRIMARY_HOST psql -U postgres -c "INSERT INTO mahasiswa (nim, nama, jurusan) VALUES ('245410097', 'Fiatri abd', 'Teknik Informatika');"
INSERT 0 3
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "SELECT * FROM mahasiswa;"
 nim | nama | jurusan
-----+-----+-----
245410097 | Fiatri abd | Teknik Informatika
215410197 | Ani lesteri | Informatika
225410097 | Citra Dewi | Informatika
(3 rows)
PS C:\Users\Lenovo> docker exec $STANDBY_HOST psql -U postgres -c "INSERT INTO mahasiswa (nim, nama, jurusan) VALUES ('281004', 'Doni', 'Tiru Komputer');"
ERROR: cannot execute INSERT in a read-only transaction
PS C:\Users\Lenovo> docker exec -it pg-primary bash
root@7c64ee0eb22:/#
```

Semua perubahan pada Master langsung disalurkan ke Replica melalui WAL (Write-Ahead Log).

Replica selalu memiliki data yang sama dengan Master (*eventually consistent*, hampir real-time).

Replikasi ini membuat database menjadi lebih aman, high availability, dan tahan gagal.